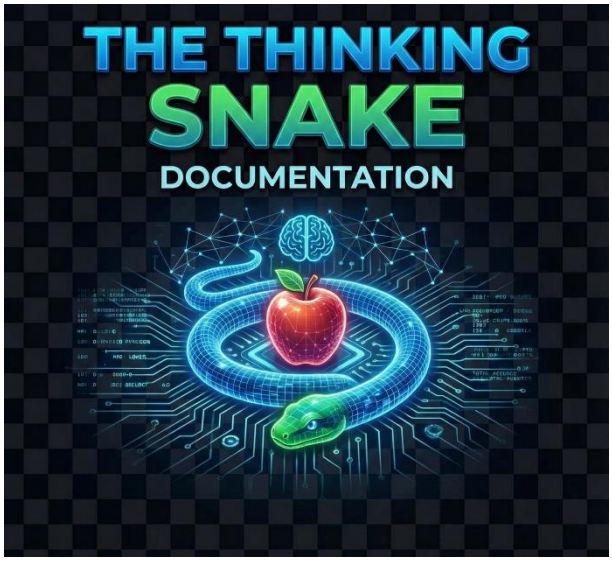
# Project Report




**Group ID : 57**

4191 : Chauhan Bhavy

4223 : Mali Dhruv

K. S. School of Business Management and Information Technology

Gujarat University

# Table of Contents

Project Report ..... 1

**The Thinking Snake** ..... 1

Group ID : 57 ..... 1

- Abstract ..... 3

1. **Introduction** ..... 5

   ❖ Introduction to the Snake Game ..... 5

   ❖ Overall Description ..... 5

   ❖ Where is it used? ..... 6

2. **Project** ..... 7

   ❖ Different ways of static path finding ..... 7

   ❖ How to apply path finding algorithms in Snake Game ..... 8

   ❖ A*(star) path Search ..... 8

   ❖ Result ..... 14

3. **Conclusion** ..... 16

   ❖ Conclusion and Future Work ..... 16

# ❖ Abstract


- **Project Overview**
  - Implements a classic Snake game enhanced with Artificial Intelligence (AI) and interactive gameplay modes.
  - Offers both manual player control and AI-driven automatic gameplay.
- **AI Integration**
  - Uses the **A*** pathfinding algorithm to compute the shortest and safest route to the food.
  - Avoids collisions with the snake's body and obstacles through efficient path evaluation.
  - Continuously recalculates optimal paths as the game environment changes.
- **Gameplay Modes**
  - **Manual Mode**
    - Player controls the snake.
    - The system evaluates player accuracy by comparing their actual path with the optimal path suggested by the AI.
  - **Automatic Mode**
    - AI agent fully controls the snake.
    - A* algorithm adapts dynamically to ensure food is reached even in complex scenarios.
- **Game Features**
  - **Dynamic Difficulty:** Game speed increases as the player progresses.
  - **Multiple Food Types:** Includes normal, golden, and poison food items.
  - **Flexible Boundary Handling:** Offers wall-collision mode or wrap-around mode.
- **Technology Used**
  - Developed using **Python**.
  - Utilizes the **PyGame** library for graphics rendering, interaction handling, and game logic.
- **Purpose and Significance**
  - Demonstrates how pathfinding algorithms can enhance classic games.
  - Provides both entertainment value and insight into algorithmic problem-solving.

# ❖ Acknowledgements


- I would like to acknowledge You tube , my supervisor, for his support and assistance throughout the duration of this project.

- I would also like to acknowledge my friends who helped by playing the game, reading the report, and offering constructive feedback.

# 1. Introduction

## ❖ Introduction to the Snake Game

- The Snake game is a classic arcade-style game that takes place in a grid-based environment.
- The key elements include the snake itself, food items, and optional obstacles or walls.
- The primary objective is for the snake to eat food, which causes its body to grow longer after each consumption.
- The snake can move in four directions: Left, Right, Up, and Down. The game ends when the snake collides with itself,
- an obstacle, or the wall (unless wrap-around mode is enabled). The final score is based on the number and type of food consumed.

## ❖ Overall Description

- This project enhances the traditional Snake game by integrating Artificial Intelligence (AI) with manual gameplay options. The player can either :
- Play manually, controlling the snake with arrow keys. An accuracy score is also displayed, showing how closely the player follows the shortest path suggested by the AI.
- Play automatically, where the snake uses the A* pathfinding algorithm to find the shortest and safest path to the food while avoiding collisions.
- **The game is further enriched with :**
- **Dynamic difficulty** : the snake's speed increases as the score goes up.
- Multiple food types : normal food (+1 score), golden food (+5 score, rare, limited time), and poison food (reduces score or shrinks snake).
- Flexible boundary modes: classic wall collision or wrap-around play.
- While other algorithms like BFS or DFS can also be applied, the A* algorithm is chosen here for its efficiency in always finding the shortest path. By continuously recalculating routes as the snake moves, it ensures adaptability even when the direct path is blocked.

## ❖ Where is it used?

- The core of this project is the application of the A* pathfinding algorithm in a simple yet interactive game environment.
- The A* algorithm is widely used in fields like robotics, maps and navigation, and AI planning systems, as it efficiently finds the shortest path between two points.
- In the context of the Snake game, it demonstrates how pathfinding can be applied to dynamic, real-time problems, where conditions change constantly (snake movement, growing body, and new food locations).

## 2. Project

- This project is an advanced implementation of the classic Snake game, designed to demonstrate the integration of Artificial Intelligence (AI) with traditional gameplay. The game allows the snake to find and eat food efficiently without colliding with walls, obstacles, or its own body.

- **The system supports two gameplay modes:**

- **Manual Mode** – The player controls the snake with arrow keys. An accuracy score is displayed to show how closely the
- player's moves align with the shortest path computed by the AI.

- **Automatic Mode** – The snake is controlled by an AI agent that uses the A* pathfinding algorithm to make decisions
- and move optimally toward the food.

- **In real-time pathfinding, two main components are involved:**

- Heading toward the goal (moving the snake toward the food).

- Avoiding obstacles dynamically (walls, the snake's body, or poison food).

- This project implements both aspects to ensure the snake can survive longer and achieve higher scores.

## ❖ Different ways of static path finding

- Pathfinding is the process of determining the optimal route between two points while avoiding obstacles.
- In the Snake game, the grid-based environment makes it suitable for applying standard search algorithms.

- Some common static pathfinding algorithms include:

- **Breadth First Search (BFS)** – Explores all nodes level by level until the goal is found. Guarantees shortest path in terms of steps but can be slow.

- **Depth First Search (DFS)** – Explores as far as possible along a path before backtracking. Not guaranteed to find the shortest path and can get stuck.

- **Dijkstra's Algorithm** – Finds the shortest path by considering cumulative costs, efficient for weighted graphs but slower in unweighted grids.

- **A* (A Star) Algorithm** – Combines the advantages of Dijkstra's and heuristic-based search. It uses a cost function (f = g + h) where:

- g = cost from the start node.

- h = estimated distance (heuristic) to the goal.

- Among these, A* is chosen in this project because it guarantees the shortest path, adapts well to real-time changes, and efficiently avoids obstacles. Unlike BFS or DFS, it can re-calculate paths dynamically when the snake's body blocks the direct route to food.

## ❖ How to apply path finding algorithms in Snake Game

Pathfinding algorithms are typically graph-based approaches. In the context of the Snake game, the environment is modeled as a grid system, where:

- Each grid cell (block) acts as a node in the graph.
- Each node can have up to four neighbors (up, down, left, right).
- The snake's head is considered the starting node, while the food location is treated as the goal node.
- When the player or the AI wants the snake to move, the system checks for possible paths from the head node to the food node. If a valid path exists, the snake can proceed safely; if not, the snake risks collision with obstacles or its own body.

## For example:

- If the snake's head is at (10, 12) and the food is at (18, 20), the algorithm evaluates all possible routes step by step.
- Each time the snake moves, the grid updates—its body becomes nebecomes new obstacles, and the algorithm recalculates the available path.
- This grid-to-graph transformation makes it possible to apply classic search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's Algorithm, and the more efficient A* Algorithm.

## ❖ A*(star) path Search

- The A* algorithm is particularly well-suited for this project because it combines the strengths of both Dijkstra's algorithm (shortest guaranteed path) and Greedy Best-First Search (fast heuristic-based decisions).

- **How it Works :**

- A* assigns a score f(n) to each possible node n that the snake could move to. This score is the sum of two components:

- g(n) → The cost of reaching node n from the start (the actual distance traveled so far).

- h(n) → A heuristic estimate of the remaining cost to reach the goal (for example, Euclidean or Manhattan distance between n and the food).

- Thus:

- $f(n)=g(n)+h(n)$
- f(n)=g(n)+h(n)

- At each step, A* chooses the node with the lowest f(n) value, meaning the one that seems most promising both in terms of distance traveled and estimated distance remaining.

## Application in Snake

- The start node is always the snake's head.

- The goal node is the food's position.

- The snake's body segments and static obstacles are treated as blocked nodes that the path cannot pass through.

- Each time the snake eats food, its body grows, and the algorithm recalculates a new path from the updated head position.

- For example:

- Snake head = (12, 15)

- Food = (18, 15)

- If there is a direct vertical path without obstacles, A* selects it (shortest path).

- If the snake's body blocks the path, A* will compute an alternate route, possibly looping around the obstacle.

## Why A* is Effective Here

- Optimality: Guarantees the shortest path if the heuristic is admissible.
- Efficiency: Much faster than exploring every possibility (like BFS or DFS).
- Dynamic Recalculation: Handles real-time changes such as the snake's growing body or new obstacles appearing.
- This ensures the AI-controlled snake can continue playing almost indefinitely, only losing when no path exists. In manual mode, player movements can also be compared against the A* computed path, allowing accuracy analysis—did the player follow the optimal route or deviate into a longer, riskier path?

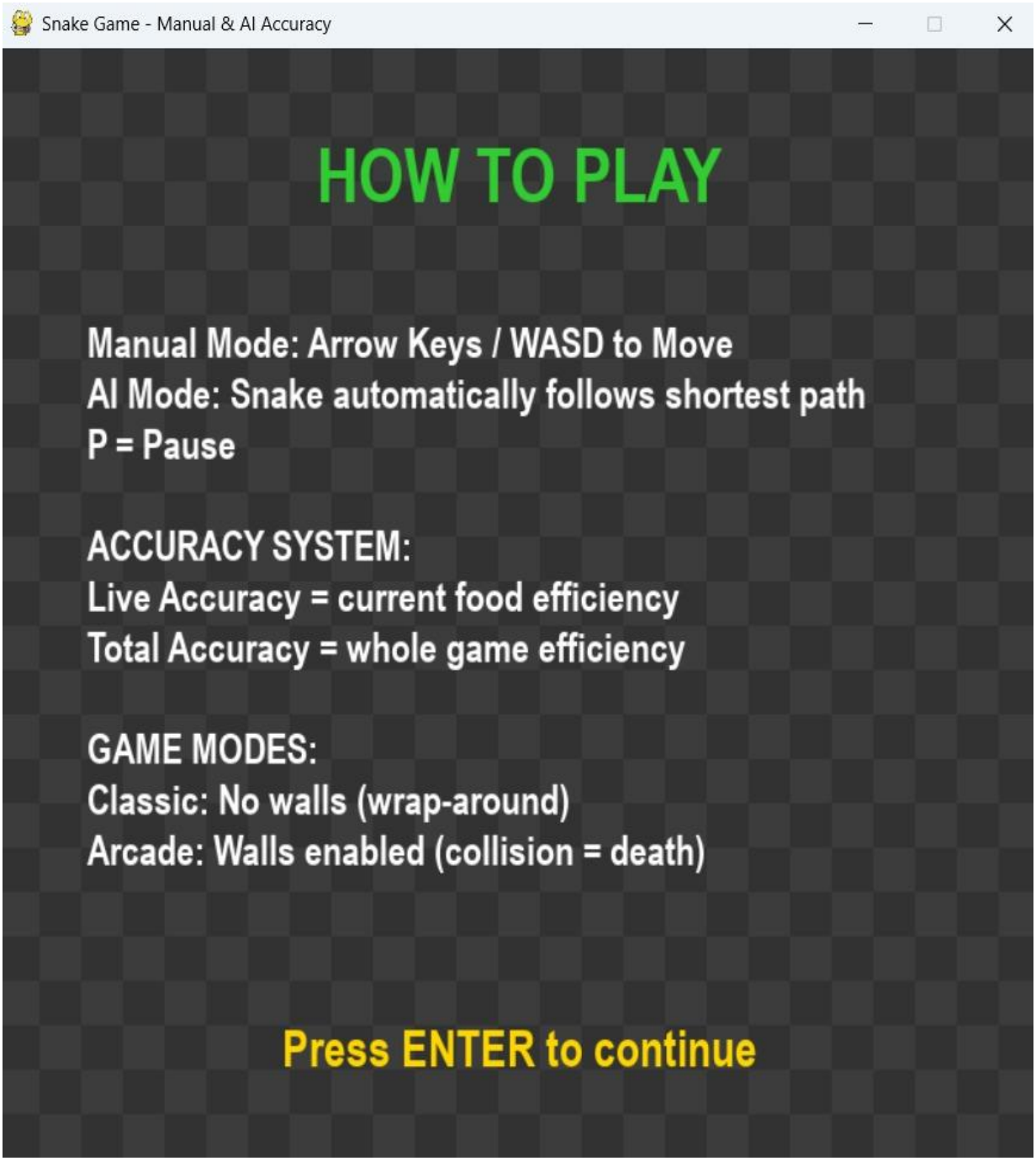
## ❖ Result

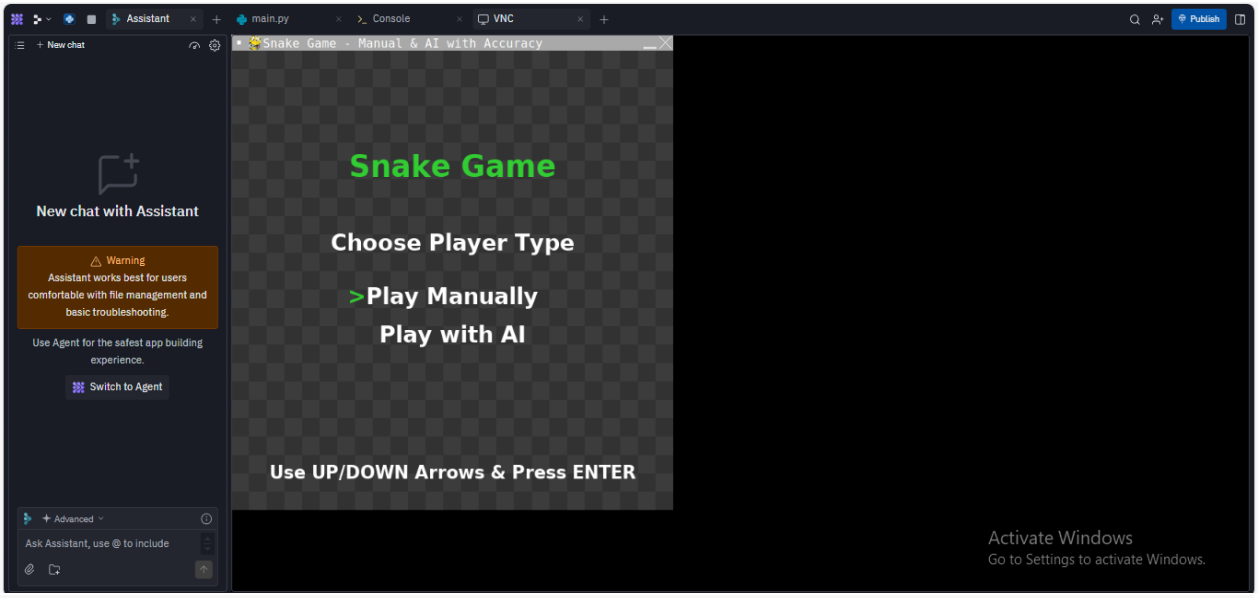
- When the program is executed, a game window titled **"Snake Game - Manual & AI with Accuracy"** appears on the screen. Inside the window, a grid of **25 rows and 25 columns** is displayed, with alternating dark and light gray squares forming the background. Several dark-colored **walls** are positioned at the corners and in the center of the grid to act as obstacles. A **blue snake** appears on the grid, starting with a small body, and a **yellow food item** is randomly placed on one of the empty cells.

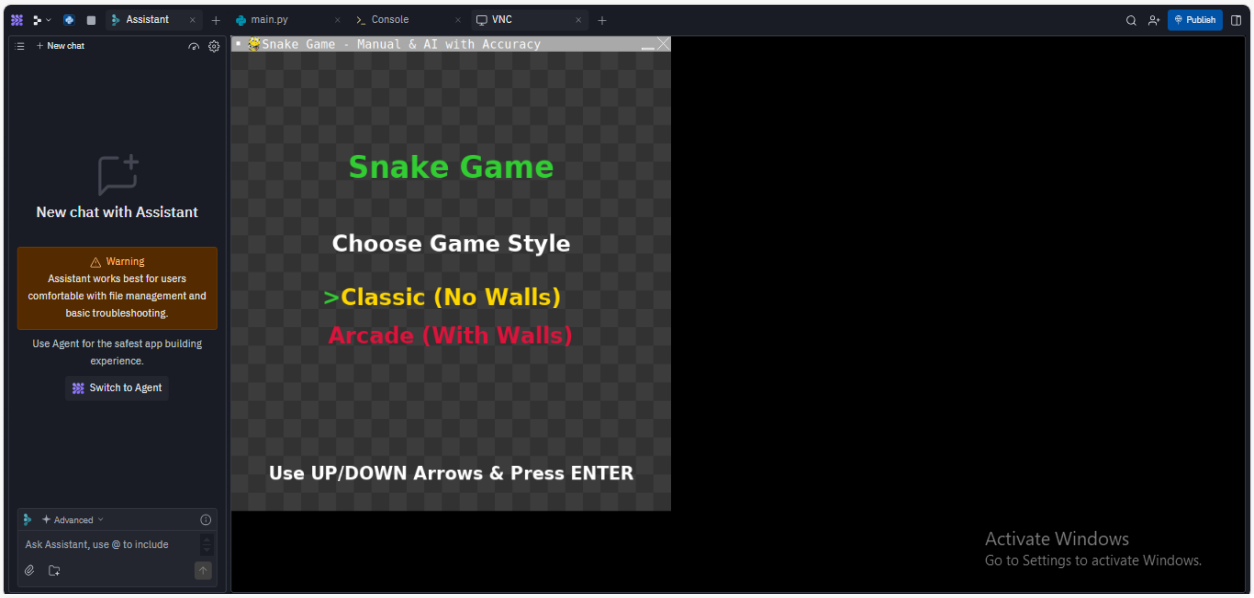
- The game can be played in two modes: **manual** and **AI-controlled**. In manual mode, the player can move the snake using the arrow keys to collect the food while avoiding collisions with the walls and the snake's own body. In AI mode, the snake automatically moves toward the food using the **A*** pathfinding algorithm, which calculates the shortest and safest path without hitting obstacles or itself. Each time the snake reaches the food, it grows longer, and a new piece of food appears in a random location.

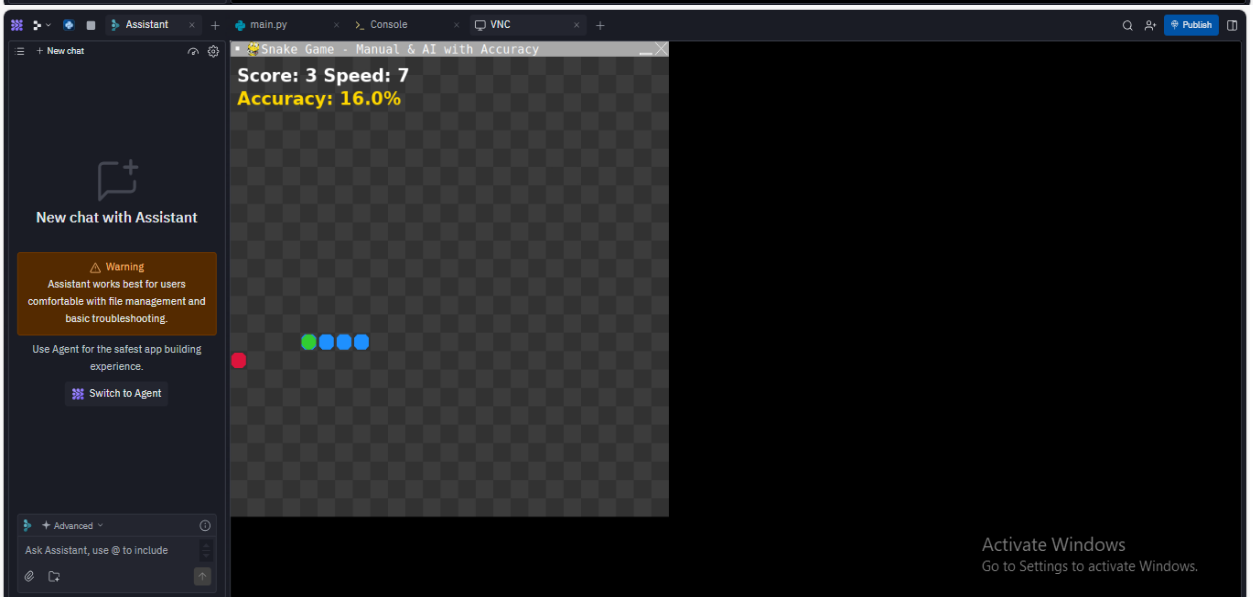
- The screen continuously updates to show the snake's movement, the score, and, in AI mode, the accuracy of the pathfinding. The game continues until the snake collides with a wall or its own body, at which point a **"Game Over"** message is displayed along with the final score. The player can then restart or close the game.

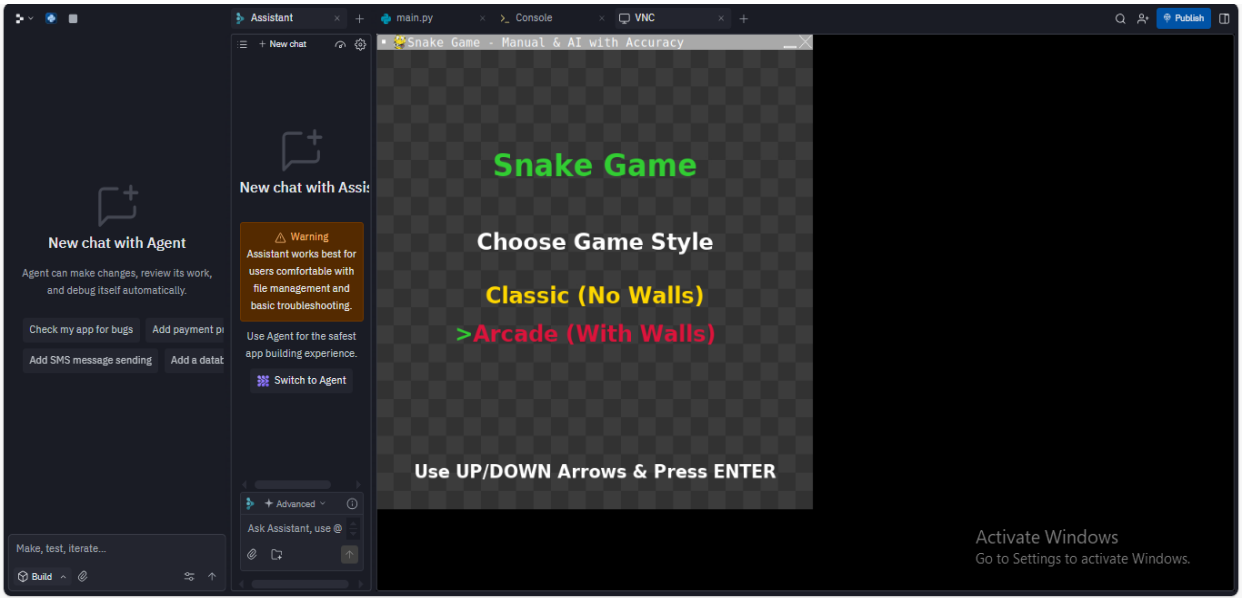
- In summary, the result of this code is an **interactive snake game** featuring both manual and AI-controlled gameplay, realistic movement, wall obstacles, scoring, and pathfinding-based automatic navigation.

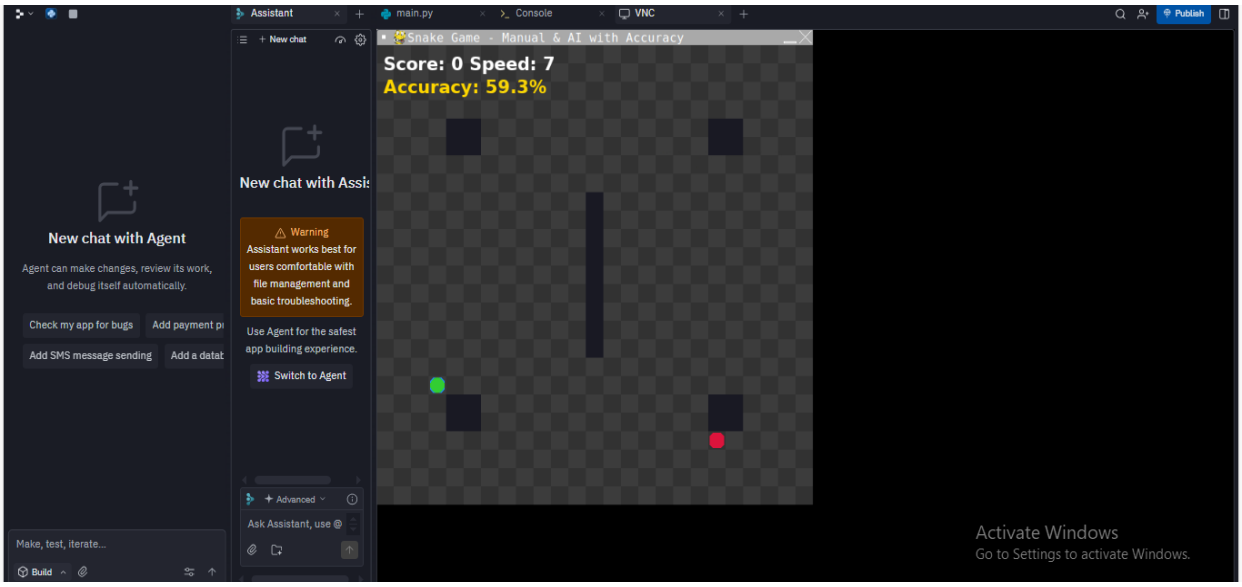

Snake Game - Manual & AI Accuracy
HOW TO PLAY
Manual Mode: Arrow Keys / WASD to Move
AI Mode: Snake automatically follows shortest path
P = Pause
ACCURACY SYSTEM:
Live Accuracy = current food efficiency
Total Accuracy = whole game efficiency
GAME MODES:
Classic: No walls (wrap-around)
Arcade: Walls enabled (collision = death)
Press ENTER to continue

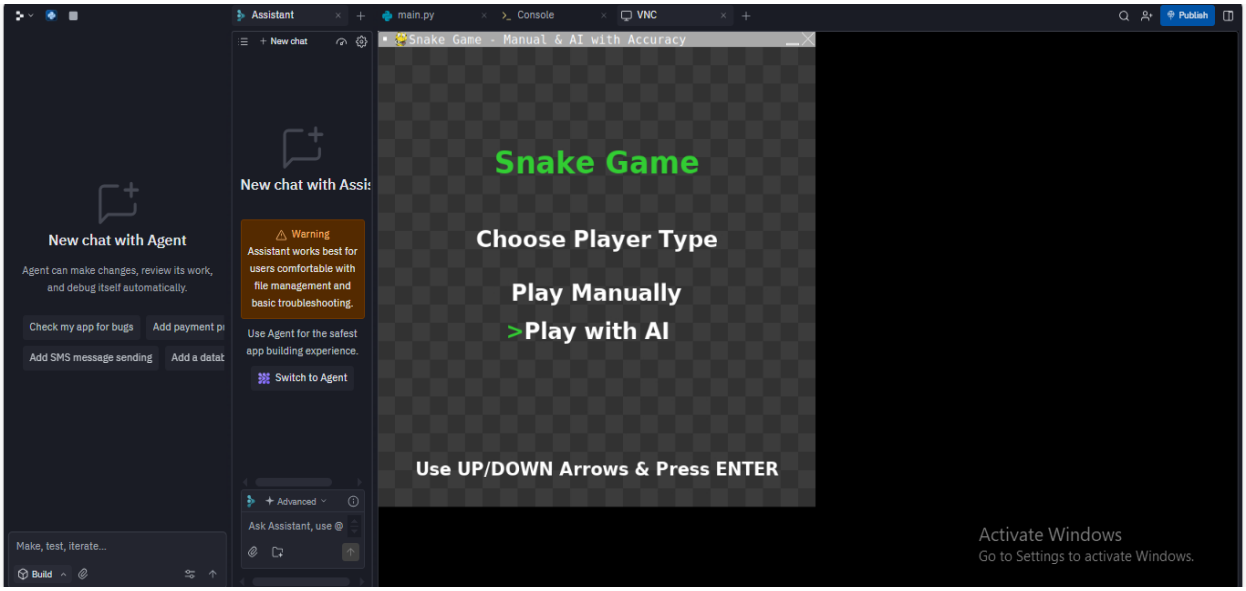

Assistant
main.py
Console
VNC
Publish
New chat
Snake Game - Manual & AI with Accuracy
New chat with Assistant
Warning
Assistant works best for users comfortable with file management and basic troubleshooting.
Use Agent for the safest app building experience.
Switch to Agent
Advanced
Ask Assistant, use @ to include
Snake Game
Choose Player Type
>Play Manually
Play with AI
Use UP/DOWN Arrows & Press ENTER
Activate Windows
Go to Settings to activate Windows.

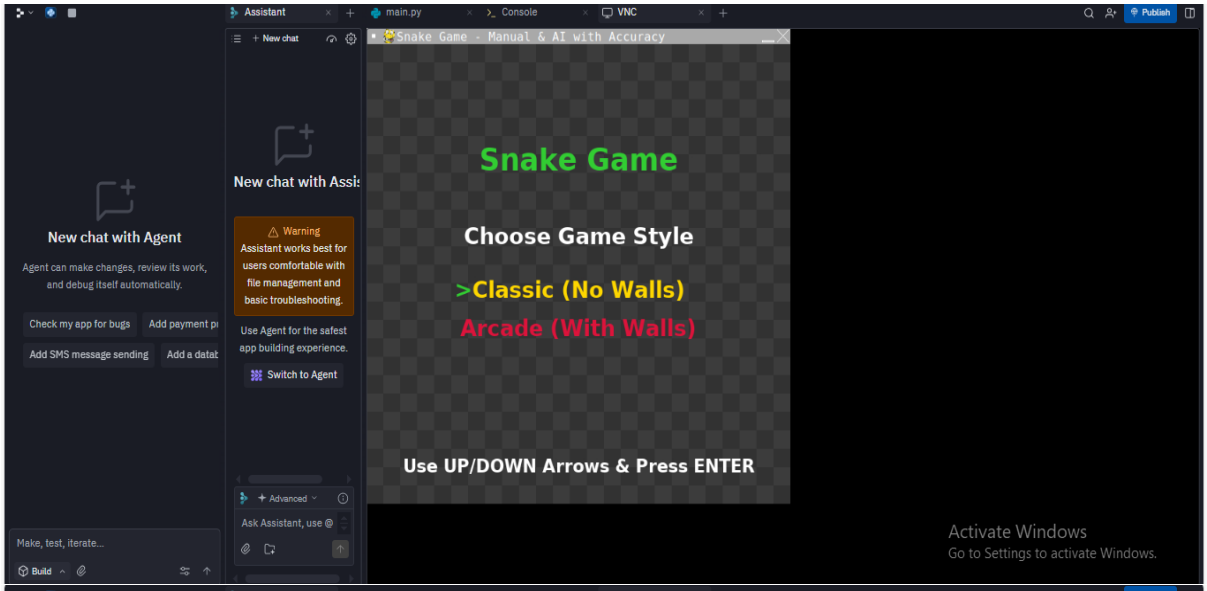

Assistant
main.py
Console
VNC
Publish
New chat
Snake Game - Manual & AI with Accuracy
New chat with Assistant
Warning
Assistant works best for users comfortable with file management and basic troubleshooting.
Use Agent for the safest app building experience.
Switch to Agent
Advanced
Ask Assistant, use @ to include
Snake Game
Choose Game Style
>Classic (No Walls)
Arcade (With Walls)
Use UP/DOWN Arrows & Press ENTER
Activate Windows
Go to Settings to activate Windows.

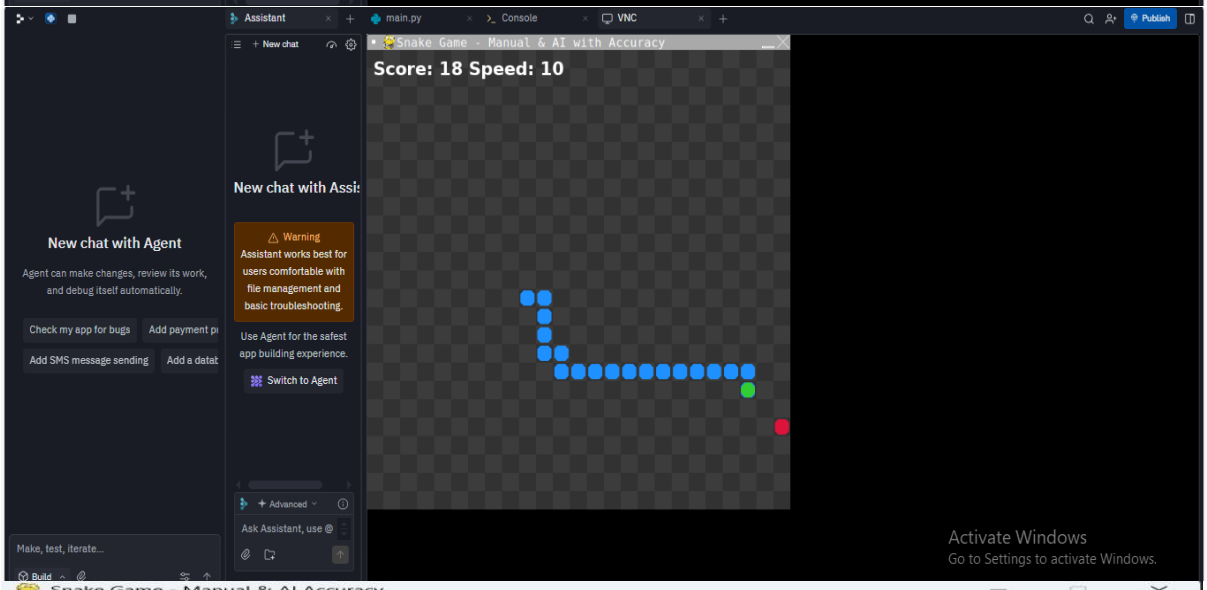

Assistant
main.py
Console
VNC
Publish
New chat
Snake Game - Manual & AI with Accuracy
Score: 3 Speed: 7
Accuracy: 16.0%
New chat with Assistant
Warning
Assistant works best for users comfortable with file management and basic troubleshooting.
Use Agent for the safest app building experience.
Switch to Agent
Advanced
Ask Assistant, use @ to include
Activate Windows
Go to Settings to activate Windows.

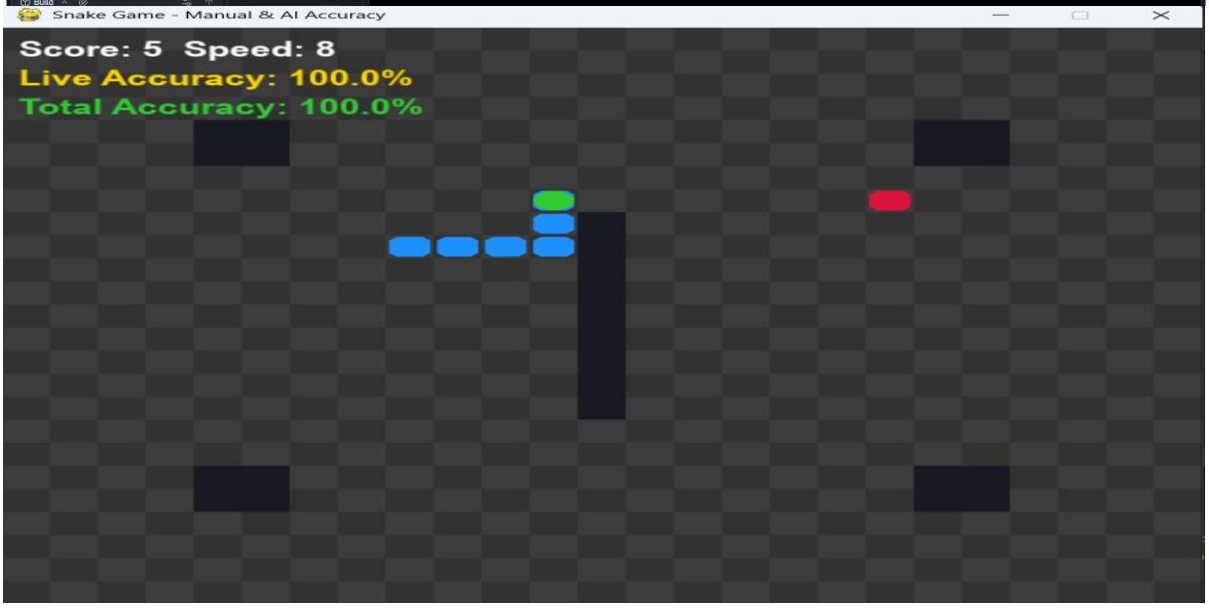

Snake Game - Manual & AI with Accuracy
Snake Game
Choose Game Style
Classic (No Walls)
>Arcade (With Walls)
Use UP/DOWN Arrows & Press ENTER
New chat with Agent
Activate Windows
Go to Settings to activate Windows.

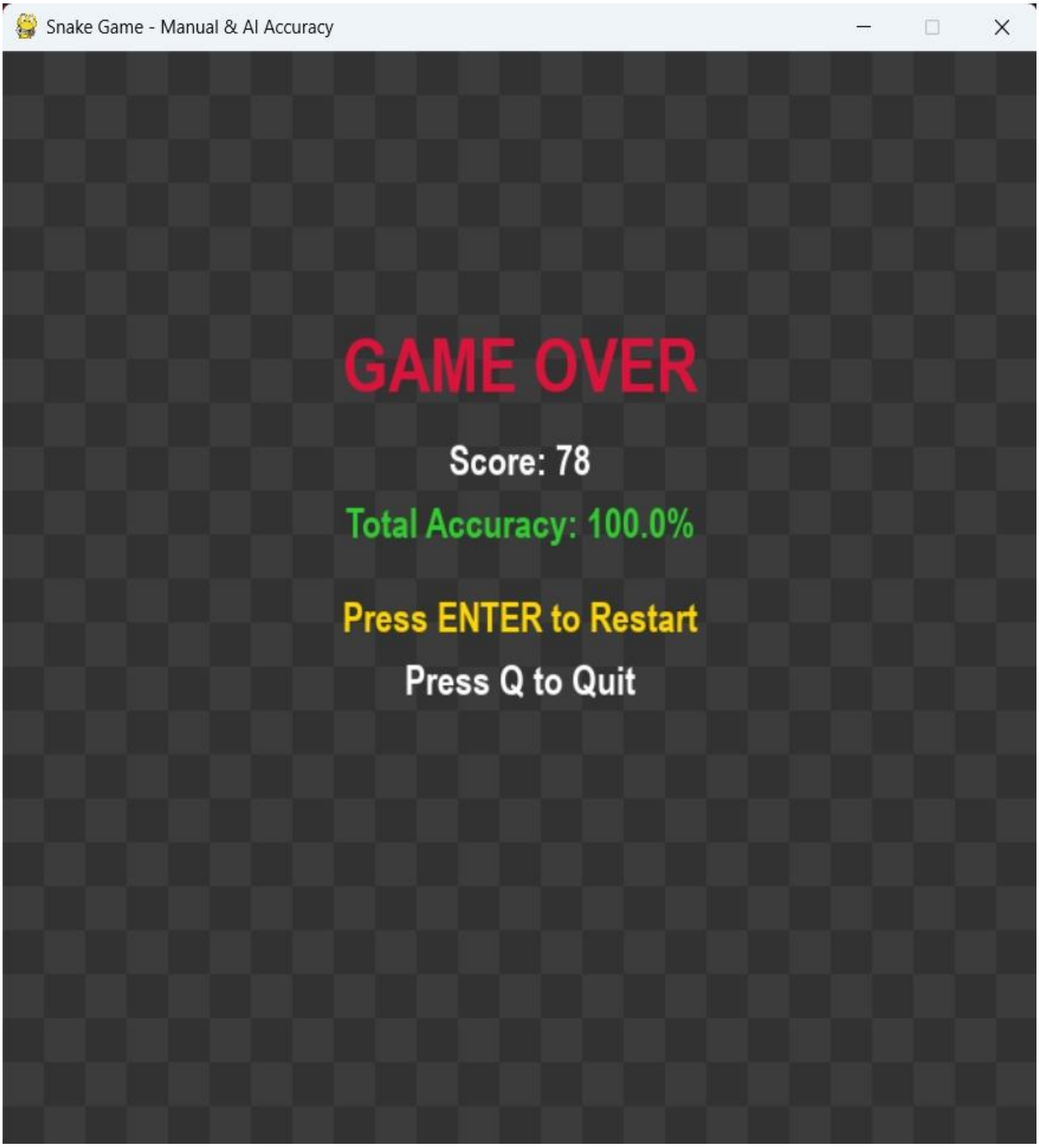

Snake Game - Manual & AI with Accuracy
Score: 0 Speed: 7
Accuracy: 59.3%
New chat with Agent
Activate Windows
Go to Settings to activate Windows.


Snake Game - Manual & AI with Accuracy
Snake Game
Choose Player Type
Play Manually
>Play with AI
Use UP/DOWN Arrows & Press ENTER
New chat with Agent
Activate Windows
Go to Settings to activate Windows.


Assistant
main.py
Console
VNC
Publish
New chat
Snake Game - Manual & AI with Accuracy
New chat with Agent
Agent can make changes, review its work, and debug itself automatically.
Check my app for bugs
Add payment pr
Add SMS message sending
Add a datat
New chat with Assi
Warning
Assistant works best for users comfortable with file management and basic troubleshooting.
Use Agent for the safest app building experience.
Switch to Agent
Advanced
Ask Assistant, use @
Make, test, iterate...
Build
Snake Game
Choose Game Style
>Classic (No Walls)
Arcade (With Walls)
Use UP/DOWN Arrows & Press ENTER
Activate Windows
Go to Settings to activate Windows.


Assistant
main.py
Console
VNC
Publish
New chat
Snake Game - Manual & AI with Accuracy
Score: 18 Speed: 10
New chat with Agent
Agent can make changes, review its work, and debug itself automatically.
Check my app for bugs
Add payment pr
Add SMS message sending
Add a datat
New chat with Assi
Warning
Assistant works best for users comfortable with file management and basic troubleshooting.
Use Agent for the safest app building experience.
Switch to Agent
Advanced
Ask Assistant, use @
Make, test, iterate...
Build
Activate Windows
Go to Settings to activate Windows.


Snake Game - Manual & AI Accuracy
Score: 5 Speed: 8
Live Accuracy: 100.0%
Total Accuracy: 100.0%


Snake Game - Manual & AI Accuracy
GAME OVER
Score: 78
Total Accuracy: 100.0%
Press ENTER to Restart
Press Q to Quit

# 3. Conclusion

- The development of this project provided valuable insights into how Artificial Intelligence (AI) techniques can be applied to classic games. By integrating the A* pathfinding algorithm with the Snake game, the project successfully demonstrated how an AI agent can make real-time decisions, avoid obstacles, and efficiently reach the goal (food).

- The system was designed with two modes:

- Manual Mode, where players control the snake and receive feedback on their accuracy compared to the AI's optimal shortest path.

- Automatic Mode, where the A* algorithm continuously recalculates the best route, ensuring that the snake can survive longer and achieve higher scores.

- Additional features such as multiple food types (normal, golden, poison), dynamic difficulty (speed increase with score), and flexible boundary modes (wrap-around or wall collision) enhanced both the challenge and the entertainment value of the game.

## ❖ Conclusion and Future Work

- The project clearly shows that pathfinding algorithms like A* can be applied successfully to dynamic environments such as the Snake game. The algorithm performed well in most scenarios, always finding the optimal route when available. However, in extreme cases where the snake's body surrounds the food, even A* cannot guarantee survival, as no valid path exists.

- **Future Improvements :**

- **Smarter AI Strategies** – Extend beyond shortest-path strategies to survival strategies, where the snake plans longer safe routes instead of always choosing the quickest option.

- **Dynamic Path Optimization** – Incorporate techniques that allow the AI to predict future deadlocks (like trapping itself) and avoid them.

- **Machine Learning / Neural Networks** – Train the snake to improve over time using reinforcement learning, rather than relying only on hardcoded pathfinding.

- **Enhanced Visuals & User Interface** – Add more engaging backgrounds, animations, and smoother transitions for a better gaming experience.

- **Multiplayer or Online Mode** – Allow two players (manual vs AI or manual vs manual) for added competitiveness.

- This project demonstrates how combining classic gameplay with AI decision-making can create an engaging and educational experience, opening the door for applying similar methods to other games and simulations.

## ❖References

1 Youtube.com

2 pygame documentation

3 githube.com

# THANK YOU